

El problema que describes es el clásico muro contra el que chocan todas las startups de *PropTech* / *ConTech*: **el sobreajuste (overfitting) de dominio**. Los modelos clásicos de visión (como U-Net) o los *zero-shot* entrenados en fotos del mundo real (DINO/SAM) fracasan en planos CAD porque los vectores (achurados, grosores de línea, cotas superpuestas, bloques dinámicos de puertas que varían por software) no son "texturas" o "bordes naturales", son un lenguaje simbólico de alta variabilidad. Cuando cambias de oficina de arquitectura, cambias de "dialecto", y los umbrales determinísticos o modelos sobreajustados se rompen.

Aquí tienes las alternativas concretas, evaluadas bajo tu filtro de viabilidad y generalización, priorizando los descubrimientos más recientes y descartando el "humo" de marketing.

1. Papers / Repositorios (2025-2026) listos para producción

La comunidad de investigación acaba de dar un giro radical en cómo aborda este problema: han abandonado la segmentación píxel a píxel (U-Net/CubiCasa) para pasar a **Vision-Language Models (VLMs) que botan JSON topológico nativo**.

El candidato principal: FloorplanVLM (Publicado en Febrero 2026)

Este es el paper/modelo que más se alinea con tu desafío de generalización. El paper ("*FloorplanVLM: A Vision-Language Model for Floorplan Vectorization*") reformula la vectorización no como segmentación de píxeles, sino como un problema de "secuencia".

- **Cómo funciona y por qué generaliza:** El modelo "lee" la imagen y emite una secuencia JSON estructurada. Primero escupe el esqueleto (muros con coordenadas, curvatura y grosor), luego anida los vanos (puertas y ventanas), y finalmente define los recintos referenciando el ID de los muros que los encierran. Al usar un VLM multimodal robusto como *backbone*, hereda una comprensión semántica enorme que ignora el "ruido CAD" (achurados, muebles falsos, cotas) que confunde a modelos antiguos.
- **Acceso / Fricción:** Open-source. El código, los pesos y su benchmark (FPBench-2K) son públicos (disponibles en HuggingFace y GitHub bajo repositorios asociados al paper, típicamente [Salesforce/Libra](#) o autores como Yulong Li / Yuanqing Liu). La fricción es infraestructura: requerirás GPUs para correr la inferencia (self-host).
- **Evidencia verificable:** El paper reporta un **92.52% de IoU en muros exteriores**. Más importante para ti: su benchmark incluye "arquitecturas no-Manhattan" (muros en diagonal, curvas, geometrías complejas), lo cual demuestra que no está sobreajustado a las típicas cajas ortogonales finlandesas o de EE. UU.
- **Rol en tu pipeline:** Reemplaza o actúa de oráculo absoluto sobre tu capa de extracción geométrica, entregando un JSON directamente consumible por tu motor de reglas de la OGUC.

2. Herramientas y APIs Comerciales (Nivel Enterprise)

Si quieres dejar de mantener modelos de visión en Colab y pagar por una API que ya resolvió el "ruido CAD", estas son las únicas opciones reales (descartando herramientas de contratistas puras como Togonal o limitadas como CubiCasa).

A. Archilogic (Space API & Floor Plan SDK)

Es el estándar de oro actual para convertir planos 2D en topología espacial, orientado cien por ciento a desarrolladores.

- **Acceso en la práctica:** Entrás a archilogic.com/developers, te registras y generas un Access Token. Su API GraphQL te entrega los polígonos, muros, puertas (con radio de apertura) y recintos en un JSON jerárquico. **Dato vital:** Acaban de lanzar (2026) un **MCP Server** (Model Context Protocol), lo que significa que tu pipeline de Claude Vision podría interactuar directamente con la data espacial de Archilogic en tiempo real.
- **Fricción (¡Cuidado aquí!):** La fricción no está en consumir la data, sino en la ingesta. Archilogic tiene un motor brutal, pero muchas veces su servicio de "Plano 2D a Modelo" involucra *human-in-the-loop* (su equipo interno) para garantizar el modelo 3D. Debes confirmar con ventas si tienen un endpoint 100% automatizado de "PDF a JSON" instantáneo para tu volumen de planos sin pasar por su servicio humano, ya que eso mata la inmediatez de tu validador.
- **Precio:** SaaS B2B. Los tiers de ingesta masiva requieren contacto de ventas, lo que probablemente exceda el presupuesto de un pequeño estudio chileno si decides traspasarles el costo por plano.

B. Motor FPR de Kreo (Floor Plan Recognition)

Aunque Kreo es un software SaaS de cubicación (similar a Togonal), publicaron a fines de 2025/2026 su nueva arquitectura híbrida para extracción.

- **Cómo funciona:** Mezclan I-OCR (Intelligent OCR) ajustado para fuentes Type3 / CAD corrupto, con un modelo de visión profundo. Están diseñados explícitamente para PDF y DWG, enfocados en separar las líneas estructurales del dibujo técnico (algo en lo que tu método de OpenCV + Bézier sufre cuando el delineante usa colores o capas raras).
- **Fricción / Acceso:** Es comercial. Tienen API, pero el registro requiere validación corporativa. Si logras acceder al entorno de desarrollador, el output es altamente preciso.

(Nota sobre CubiCasa 2.0: Descártalo como API de ingesta. "CubiCasa 2.0" fue un update de producto enfocado en LiDAR y escaneo con smartphone de áreas totales (GIA/GLA), no una API abierta para subir PDFs 2D de terceros. Su negocio es que los fotógrafos usen su app in situ).

3. Recomendación Priorizada para ArchiCheck

Dado lo que ya construiste (que es un híbrido muy inteligente entre OpenCV determinístico y LLMs como Claude Vision) y sabiendo que tu dolor crítico es la **falta de estandarización en el dibujo CAD chileno**, esto es lo que debes probar:

Prioridad 1: Testear FloorplanVLM (El camino del LLM topológico).

- **Por qué:** Tu intuición de usar Claude Vision iba por el camino correcto (semántica), pero Claude Vision no bota coordenadas precisas. FloorplanVLM es la unión perfecta: tiene la resiliencia semántica para "entender" un plano sin importar los achurados, y la arquitectura técnica para botar el esqueleto geométrico vectorizado en JSON. Si logras levantar los pesos de este modelo, resolverás el problema de las convenciones de dibujo distintas en un solo paso, porque el modelo fue entrenado prediciendo dependencias (muros -> recintos), no pintando píxeles.

Prioridad 2: Actualizar tu pipeline determinístico inyectando el concepto de "Space-Partitioning" basado en centroides (Paper: Comprehensive floor plan vectorization, 2025).

- **Por qué:** Si no puedes levantar FloorplanVLM, no abandones tu código OpenCV. Gran parte del fracaso determinístico de puertas/muros ocurre al procesar *líneas en el vacío*. La literatura de 2025 demuestra que si primero detectas el "Centro Geométrico" (el vacío del recinto) usando algoritmos de segmentación regional insensibles al achurado, y luego expandes la búsqueda topológica hacia afuera (Quadrant Cone Search) para encontrar los muros, evitas que una puerta dibujada con una convención extraña rompa la conectividad del muro.
- **Impacto:** Te permite usar tu código existente pero haciendo que el recinto defina el contorno, en lugar de que el contorno defina el recinto.

Prioridad 3: Archilogic API (Como Benchmark de Ground Truth).

- Crea una cuenta de desarrollador gratuita. Sube a través de su plataforma (aunque demore 24 horas por validación) tus planos chilenos más problemáticos (esos 7 donde no aparecen las puertas). Usa su GraphQL API para extraer el JSON resultante y compara esa topología perfecta contra lo que extrae tu pipeline en Colab. Esto te dará el *target* JSON ideal y te permitirá diagnosticar si el problema de tu OpenCV es de detección, o de lógica de agrupamiento.